

Exercise

A blood-based mRNA signature distinguishes people with Long COVID from recovered individuals

Presentation



Frontiers in Immunology

TYPE Original Research
PUBLISHED 03 December 2024
DOI 10.3389/fimmu.2024.1450853



OPEN ACCESS

EDITED BY

Fatemeh Saheb Sharif-Askari,
University of Sharjah, United Arab Emirates

REVIEWED BY

Stelvio Tonello,
University of Eastern Piedmont, Italy
Juliana Gonçalves,
Human Immunobiology and Pathogenesis
Laboratory, Portugal

*CORRESPONDENCE

Sarah J. Annesley
✉ S.Annesley@latrobe.edu.au

[†]These authors have contributed
equally to this work and share
first authorship

RECEIVED 18 June 2024

ACCEPTED 13 November 2024

PUBLISHED 03 December 2024

CITATION

A blood-based mRNA signature distinguishes people with Long COVID from recovered individuals

Daniel Missailidis^{1†}, Esmaeil Ebrahimie^{2,3†},
Manijeh Mohammadi Dehcheshmeh², Claire Allan¹,
Oana Sanislav¹, Paul Fisher¹, Stephanie Gras^{4,5,6}
and Sarah J. Annesley^{1*}

¹Department of Microbiology, Anatomy, Physiology and Pharmacology, La Trobe University, Bundoora, VIC, Australia, ²Genomics Research Platform, School of Agriculture, Biomedicine and Environment, La Trobe University, Bundoora, VIC, Australia, ³School of Animal and Veterinary Sciences, Faculty of Sciences, Engineering and Technology, University of Adelaide, Adelaide, SA, Australia, ⁴Infection & Immunity Program, La Trobe Institute for Molecular Science (LIMS), La Trobe University, Bundoora, VIC, Australia, ⁵Department of Biochemistry and Chemistry, School of Agriculture, Biomedicine and Environment, La Trobe University, Bundoora, VIC, Australia, ⁶Department of Biochemistry and Molecular Biology, Monash University, Clayton, VIC, Australia

Task 1

Read in data

- count table
- annotation
- tpm data

```
print(count_table.head())
```

gene_id	SRR31535847	SRR31535848
A1BG	231.000	250.000
A1BG-AS1	26.900	5.304
A1CF	0.000	0.000
A2M	10.000	0.000
A2M-AS1	3.000	25.000

```
print(clinical_data.head())
```

	sampleName	sampleID	Condition
0	vacSG102	SRR31535849	CR
1	vacSG105	SRR31535848	CR
2	vacSG111	SRR31535847	CR
3	vacSG112	SRR31535858	CR
4	vacSG70	SRR31535850	CR

```
print(tpm_table.head())
```

gene_name	SRR31535847	SRR31535848
gene_id		
A1BG	A1BG	9.289695
A1BG-AS1	A1BG-AS1	0.867618

Task 1 - Solution

Read in data

- count table
- annotation
- tpm data

```
print(count_table.head())
```

gene_id	SRR31535847	SRR31535848
A1BG	231.000	250.000
A1BG-AS1	26.909	5.304
A1CF	0.000	0.000
A2M	10.000	0.000
A2M-AS1	3.000	25.000

```
print(tpm_table.head())
```

	gene_name	SRR31535847	SRR31535848
gene_id			
A1BG	A1BG	9.289695	10.663707
A1BG-AS1	A1BG-AS1	0.867618	0.181754

```
import pandas as pd
```

```
file_path = "Data/salmon.merged.gene_counts.tsv"
```

```
# Read in count file
```

```
count_table = pd.read_csv(file_path, sep='\t', index_col=0)
```

```
count_table = count_table.drop('gene_name', axis=1)
```

```
# Get an overview of the file
```

```
print(count_table.head())
```

```
file_path_cl = "Data/srs_srr_name.csv"
```

```
# Read in file
```

```
clinical_data = pd.read_csv(file_path_cl, sep=';', skip_blank_lines=True)
```

```
# Get an overview of the file
```

```
print(clinical_data.head())
```

```
file_path = "Data/salmon.merged.gene_tpm.tsv"
```

```
# Read in tpm file
```

```
tpm_table = pd.read_csv(file_path, sep='\t', index_col=0)
```

```
tpm_table.head()
```

Task 2

Preparation for DESeq

- count table: columns = genes; rows = samples (as row names)
 - filtered for lowly expressed genes
- metadata: columns = group; rows = samples (as row names)

Task 2 - Solution

Preparation for DESeq

- count table

- metadata

```
# filter for lowly expressed genes
genes_to_keep = count_table.columns[count_table.sum(axis=0) >= 10]
counts_df = count_table[genes_to_keep]

# convert dataframe to integer
counts_df = counts_df.astype(int)

# transpose dataframe for Deseq analysis
counts_df = counts_df.T

counts_df.head()
```

```
# Extract count matrix column names
sample_names = count_table.columns
sample_names

# Reorder to match counts_df columns
group_info = clinical_data.set_index('sampleID').loc[sample_names]

# Create new DataFrame linking sample to group
metaData = pd.DataFrame({
    "Sample": sample_names,
    "Group": group_info["Condition"].values
})

# set sample as index
metaData.set_index("Sample", inplace=True)

print(metaData)
```

Task 3

Running DESeq2

- create DeseqDataSet
- run `dds.deseq2()`
- DeseqStats ds & summary

Task 3 - Solution

Running DESeq2

- create DeseqDataSet
- run dds.deseq2()
- DeseqStats ds & summary

```
from pydeseq2.dds import DeseqDataSet
from pydeseq2.ds import DeseqStats

# create DeseqDataSet from counts and sample information
dds = DeseqDataSet(
    counts=counts_df,
    metadata=metaData,
    design="~Group",
    refit_cooks=True)
```

```
# fit Deseq model
dds.deseq2()
```

```
# calculate DESeq
ds = DeseqStats(dds, contrast=["Group", "LC", "CR"], n_cpus=1)

# get ds summary overview
ds.summary()
```


Task 4

Get significantly expressed genes

- filter upregulated genes
- filter downregulated genes

Task 4 - Solution

Get significantly expressed genes

- filter upregulated genes
- filter downregulated genes

```
# get results in object
res = ds.results_df

# get significantly upregulated genes
sigUp = res[(res["padj"] < 0.05) & (res["log2FoldChange"] > 0)]

# get significantly downregulated genes
sigDown = res[(res["padj"] < 0.05) & (res["log2FoldChange"] < 0)]
```

```
# select the upregulated genes in tpm table
tpm_up = tpm_table.loc[sigUp.index.tolist()]

# drop the column gene_name
tpm_up.drop('gene_name', axis = 1, inplace = True)
tpm_up.head()

# select the downregulated genes in tpm table
tpm_down = tpm_table.loc[sigDown.index.tolist()]

# drop the column gene_name
tpm_down.drop('gene_name', axis = 1, inplace = True)
tpm_down.head()
```

Task 5

Prepare data for heatmap

- Map columns to sample names
- reorder columns
- create final, merged data

Task 5 - Solution

Prepare data for heatmap

- Map columns to sample names
- reorder columns
- create final, merged data

```
# change sample names
mapping = dict(zip(clinical_data["sampleID"], clinical_data["sampleName"]))
tpm_up = tpm_up.rename(columns=mapping)
tpm_down = tpm_down.rename(columns=mapping)

# get the order of the samples according to the plot in the paper
desired_order = ["SG22", "vacSG60", "vacSG116", "vacSG117", "vacSG47", "vacSG55", "vacSG61",
                  "vacSG102", "vacSG105", "vacSG111", "vacSG21", "vacSG34", "vacSG70", "vacSG112"]

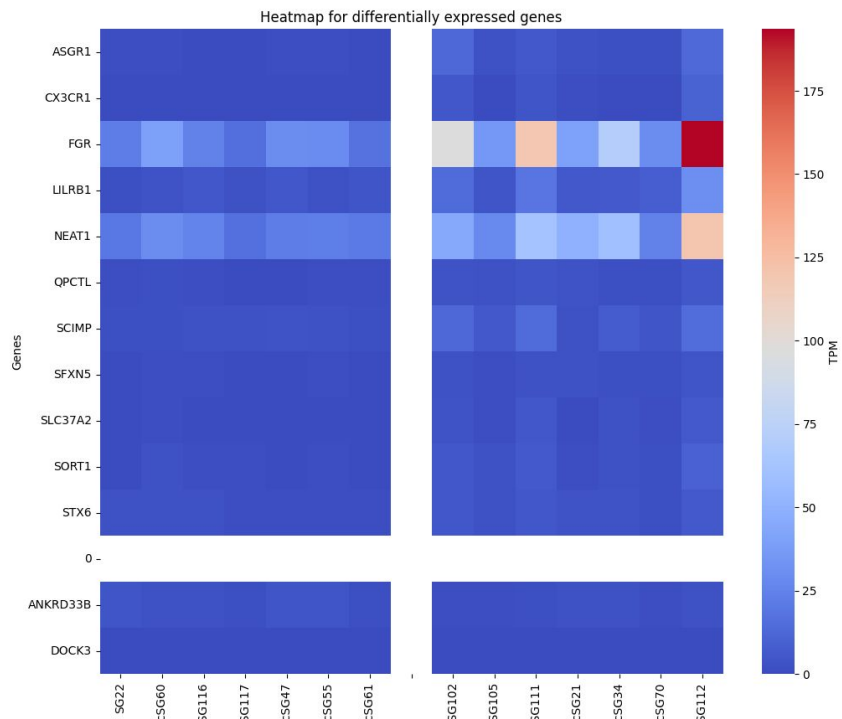
# reorder the data
tpm_up = tpm_up[desired_order]
tpm_down = tpm_down[desired_order]

# add an NAN row for the blank
nan_row = pd.DataFrame([[np.nan]*tpm_down.shape[1]], columns=tpm_down.columns)
tpm_sig_naRow = pd.concat([tpm_down, nan_row, tpm_up])

# add an NAN column for the blank
tpm_sig_gap = tpm_sig_naRow.copy()
tpm_sig_gap.insert(7, " ", np.nan) # position index starts at 0
```

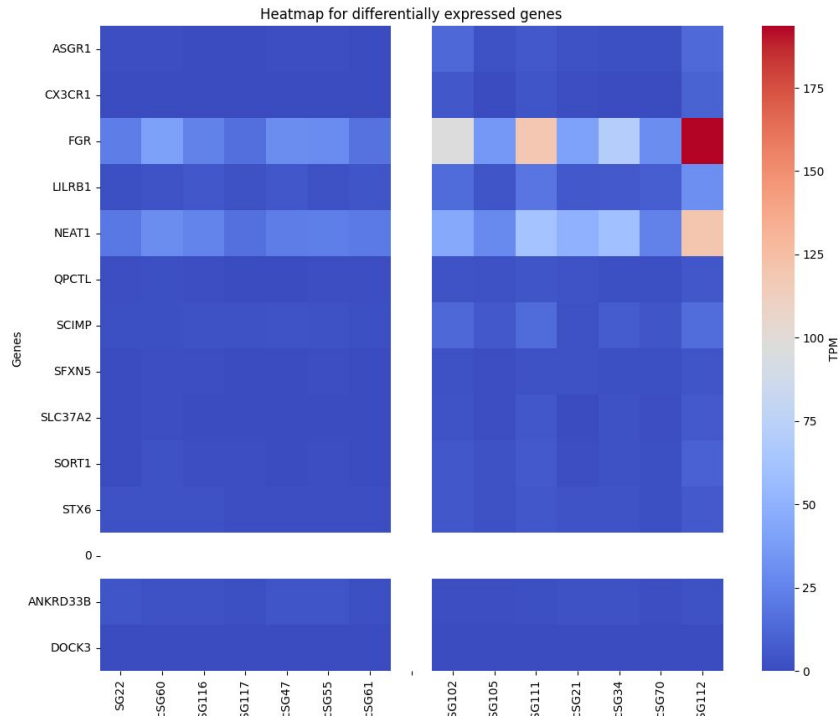
Task 6

Plot heatmap



Task 6 - Solution

Plot heatmap



```
import seaborn as sns

plt.figure(figsize=(12,10))
# plot heatmap
sns.heatmap(
    tpm_sig_gap,
    cmap="coolwarm",           # red = high, blue = low
    yticklabels=True,
    xticklabels=True,
    cbar_kws={'label': 'TPM'}
)
plt.title("Heatmap for differentially expressed genes")
plt.ylabel("Genes")
plt.xlabel("Samples")
plt.show()
```

Task 7

Prepare for PCA

- combine up&downregulated genes
 - columns = genes; rows = samples
- scale the values
- run PCA

Task 7 - Solution

Prepare for PCA

- combine up&downregulated genes
 - columns = genes; rows = samples
- scale the values
- run PCA

```
# get significant gene dataframe and transpose it for PCA  
tpm_sig = pd.concat([tpm_down, tpm_up]).T  
tpm_sig.head()
```

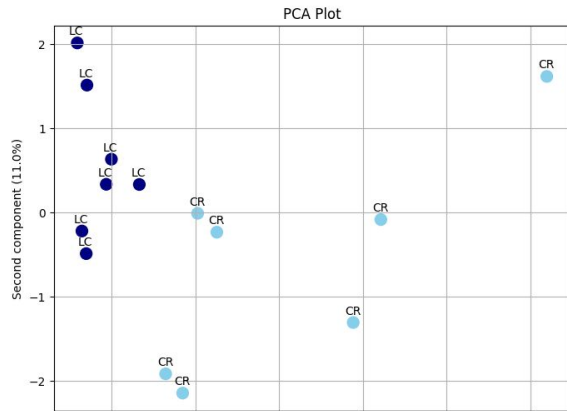
```
from sklearn.decomposition import PCA  
from sklearn.preprocessing import StandardScaler  
  
# scale the values  
X = tpm_sig.values  
tpm_sig_stand = StandardScaler().fit_transform(X)
```

```
# calculate PCA with 2 components  
pca = PCA(n_components=2) # first 2 principal components  
principalComponents = pca.fit_transform(tpm_sig_stand)  
  
# Convert to DataFrame for plotting  
pca_df = pd.DataFrame(data=principalComponents, columns=['PC1', 'PC2'])
```


Task 8

Plot PCA

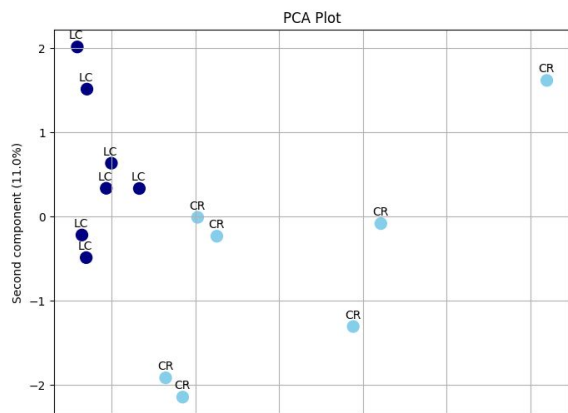
- create data including relevant information
- plot PCA scatterplot



Task 8 - Solution

Plot PCA

- create data including relevant information
- plot PCA scatterplot



```
# Convert to DataFrame for plotting
pca_df = pd.DataFrame(data=principalComponents, columns=['PC1', 'PC2'])

#add sample names
pca_df['Sample'] = tpm_sig.index

#add group and delete duplicated column
merged = pd.merge(pca_df, clinical_data, left_on="Sample", right_on="sampleName", how="inner")
merged = merged.drop(columns="sampleName")

# map group to colors
colors = {'LC':'navy', 'CR':'skyblue'}

plt.figure(figsize=(8,6))
# make scatterplot from PC1 and PC2, colored in condition
plt.scatter(merged['PC1'], merged['PC2'], c=merged['Condition'].map(colors), s=100)
# add x and y Label (formatted string including explained variance ratio)
plt.xlabel(f"First component ({pca.explained_variance_ratio_[0]*100:.1f}%)")
plt.ylabel(f"Second component ({pca.explained_variance_ratio_[1]*100:.1f}%)")
# include Labels for each point
for i in range(merged.shape[0]):
    plt.text(merged['PC1'][i]-0.2, merged['PC2'][i]+0.1, merged['Condition'][i], fontsize=10)
# add title
plt.title("PCA Plot")
plt.grid(True)
plt.show()
```

Task 9

Expression comparison

S1PR1

- create data from S1PR1 expression per sample
- get p-value from difference LC/CR

Task 9 - Solution

Expression comparison S1PR1

- create data from S1PR1 expression per sample
- get p-value from difference LC/CR

```
# plot S1PR1, therefor extract the TPM values for that gene
tpm_S1PR1 = tpm_table.loc["S1PR1"]
tpm_S1PR1
# merge the values with the clinical data
tpm_S1PR1_merged = pd.merge(tpm_S1PR1, clinical_data,
                             left_index=True, right_on="sampleID", how="inner")
tpm_S1PR1_merged

from scipy import stats

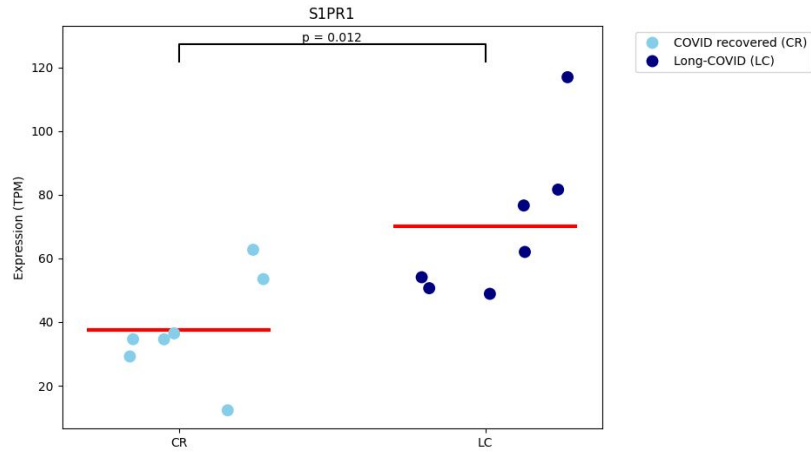
# Force conversion to numeric
tpm_S1PR1_merged['S1PR1'] = pd.to_numeric(tpm_S1PR1_merged['S1PR1'], errors='coerce')

# make two groups for statistical test
group1 = tpm_S1PR1_merged[tpm_S1PR1_merged['Condition']=='CR']['S1PR1']
group2 = tpm_S1PR1_merged[tpm_S1PR1_merged['Condition']=='LC']['S1PR1']

# test for difference in mean (T-test)
t_stat, p_val = stats.ttest_ind(group1, group2)
print("p-value:", p_val)
```

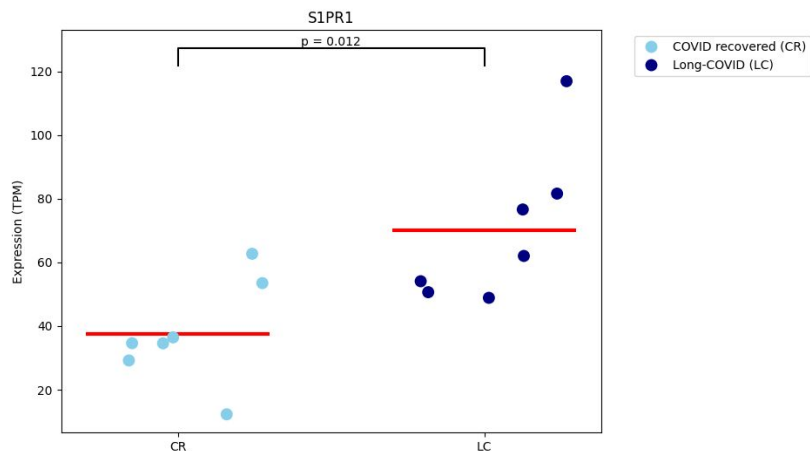
Task 10

Plot expression comparison
S1PR1



Task 10 - Solution

Plot expression comparison S1PR1



```
plt.figure(figsize=(8,6))
# make stripplot with S1PR1 expression values
sns.stripplot(x='Condition',y='S1PR1',data=tpm_S1PR1_merged,hue='Condition',
              palette=colors,jitter=0.3,dodge=False,size=10, legend=True)

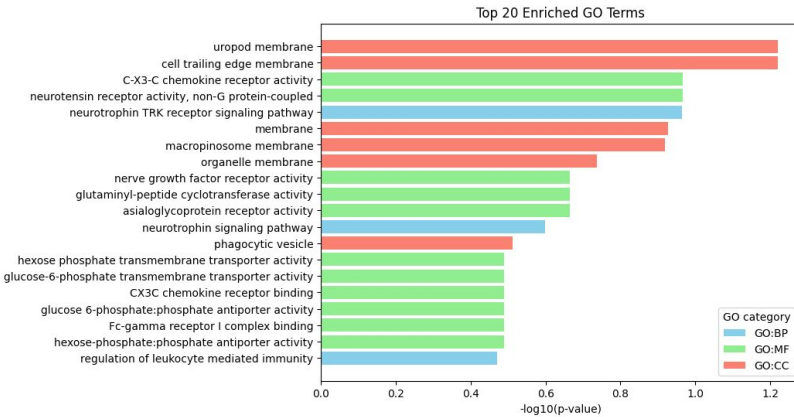
#include meanbar for each group
bar_width = 0.3
for xi, group in enumerate(tpm_S1PR1_merged.groupby('Condition')):
    mean_y = group[1]['S1PR1'].mean()
    plt.hlines(y=mean_y, xmin=xi-bar_width, xmax=xi+bar_width, color='red', linewidth=3)

# Add significance annotation
x1, x2 = 0, 1 # positions of the groups on x-axis
y, h, col = tpm_S1PR1_merged['S1PR1'].max() + 10, 0.3, 'k' # y-position, height of Line, color
plt.plot([x1, x1, x2, x2], [y-5, y+h, y+h, y-5], lw=1.5, c=col) # draw bracket
plt.text((x1+x2)*.5, y+h, f"p = {p_val:.3f}", ha='center', va='bottom', color=col)

plt.ylabel("Expression (TPM)")
plt.xlabel("")
plt.title("S1PR1")
plt.grid(False)
legend = plt.legend(bbox_to_anchor=(1.05,1), loc=2)
legend.get_texts()[0].set_text("COVID recovered (CR)")
legend.get_texts()[1].set_text("Long-COVID (LC)")
plt.show()
```

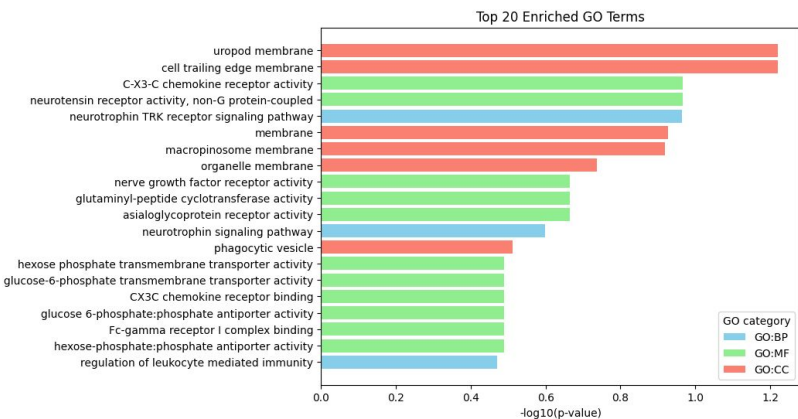
Task 11

GO analysis



Task 11 - Solution

GO analysis



```
from gprofiler import GProfiler
from matplotlib.patches import Patch
```

```
#####
##### TASK 11 #####
# GO analysis #
#####
```

```
# get names of significantly expressed genes
sigGenesAll = res[res["padj"] < 0.05]
sigGenesAll = sigGenesAll.index
sigGenesAll
```

```
#initialize GProfiler
gp = GProfiler(return_dataframe=True)
```

```
# Run enrichment for human genes
```

```
go_results = gp.profile(
    organism='hsapiens',
    query=list(sigGenesAll),
    sources=['GO:CC', 'GO:BP', 'GO:MF'],
    user_threshold=0.05,
    all_results=True
)
print(go_results[['source', 'name', 'p_value', 'term_size', 'query_size', 'intersection_size']].head(10))
```

```
# get the top 20 results from GO analysis
top_terms = go_results.head(20)
```

```
# set different colors for GO:BP, GO:MF, GO:CC
```

```
colors = {
    'GO:BP': 'skyblue',
    'GO:MF': 'lightgreen',
    'GO:CC': 'salmon'
}
```

```
# plot -log10(pvalue) and name of GO term with color-coded bars
bar_colors = [colors[src] for src in top_terms['source']]
```

```
plt.figure(figsize=(8,6))
plt.barh(
    top_terms['name'],
    -np.log10(top_terms['p_value']),
    color=bar_colors
)
```

```
plt.xlabel('-log10(p-value)')
plt.title('Top 20 Enriched GO Terms')
plt.gca().invert_yaxis() # Largest bar on top
```

```
# Add Legend manually
legend_elements = [Patch(facecolor=c, label=l) for l,c in colors.items()]
plt.legend(handles=legend_elements, title='GO category')
```

```
plt.show()
```